Library Management System (Postgres SQL Project)

Database Design • CRUD • CTAS • Advanced Analysis

This project works like a mini real-world library backend — clean schema + practical business questions.

Made with GAMMA

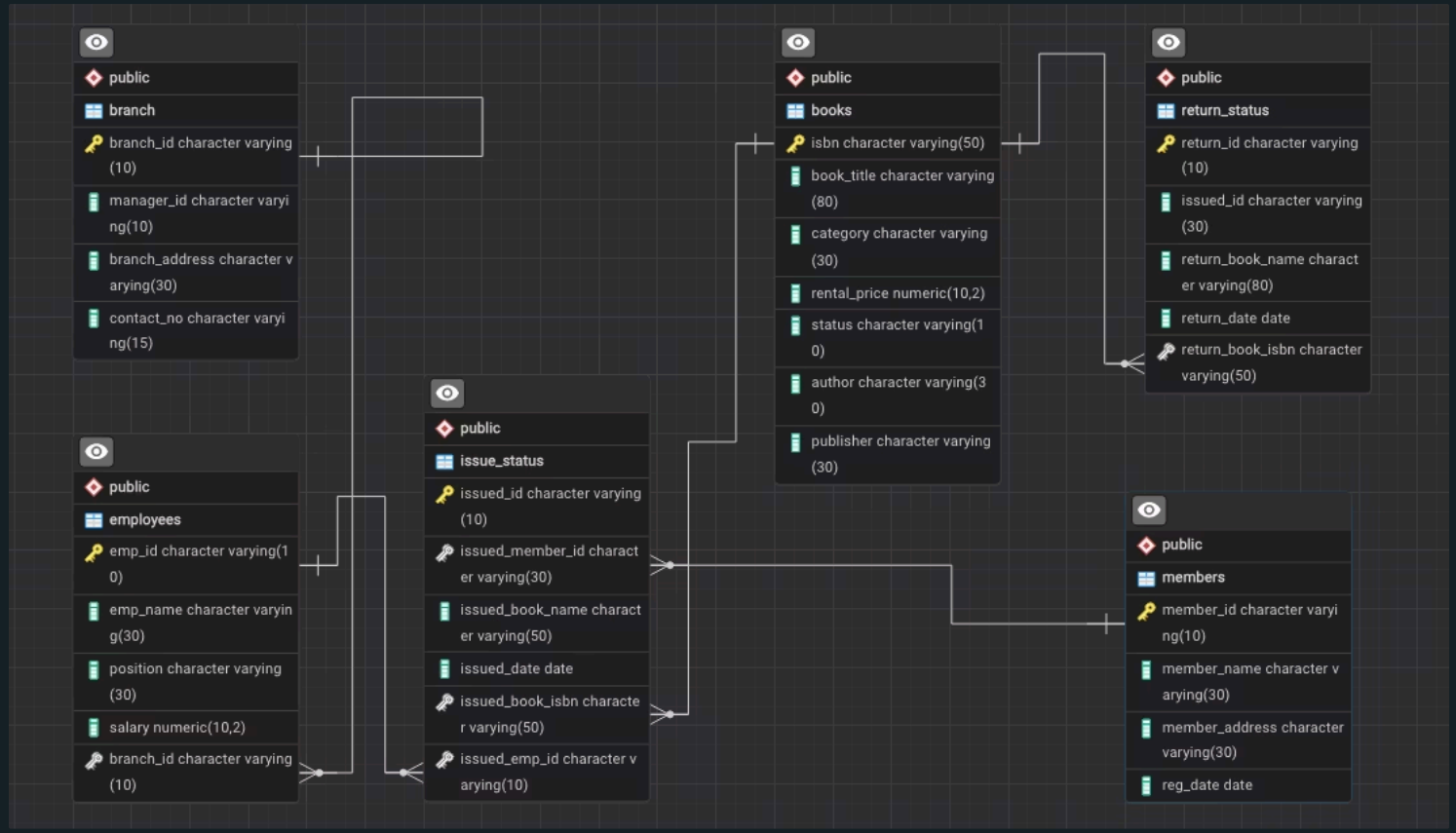
Project Overview

- Built `library_db` with core entities (branch, employees, members, books, issue/return).
- Performed CRUD operations + reporting using CTAS.
- Ran analytical queries: revenue, overdue tracking, top staff, active members.

I didn’t just write SQL queries — I modeled how a library actually runs using data.

Made with GAMMA

ERD / Data Model



Logic :

- Primary keys uniquely identify each record.
- Foreign keys connect tables and prevent orphan data.

The ERD became my story map: who borrowed what, handled by whom, and from which branch.

Made with GAMMA

Create Database

```
CREATE DATABASE library_db;
```

- Creates the main container for all tables.
- Keeps project objects organized.

One database = one world. This is where the library system starts.

Made with GAMMA

Create Tables

Create Table: branch

```sql
DROP TABLE IF EXISTS branch;

CREATE TABLE branch (
    branch_id      VARCHAR(10) PRIMARY KEY,
    manager_id     VARCHAR(10),
    branch_address VARCHAR(30),
    contact_no     VARCHAR(15)
);
```

- Stores branch identity + contact details.
- `branch_id` is the anchor key.

Branches are the locations — every operation ties back to them.

Create Table: employees

```sql
DROP TABLE IF EXISTS employees;

CREATE TABLE employees (
    emp_id    VARCHAR(10) PRIMARY KEY,
    emp_name  VARCHAR(30),
    position  VARCHAR(30),
    salary    DECIMAL(10,2),
    branch_id VARCHAR(10),
    FOREIGN KEY (branch_id) REFERENCES
branch(branch_id)
);
```

- Employees are linked to branches using `branch_id`.
- FK ensures every employee belongs to a valid branch.

This table helps me measure operational work by staff and by branch.

Create Table: members

```sql
DROP TABLE IF EXISTS members;

CREATE TABLE members (
    member_id      VARCHAR(10) PRIMARY KEY,
    member_name    VARCHAR(30),
    member_address VARCHAR(30),
    reg_date       DATE
);
```

- Stores member details + registration timeline.
- `reg_date` supports “recent member” analysis.

Members are the growth side of the system — activity starts here.

Made with GAMMA

Create Table: books

```sql
DROP TABLE IF EXISTS books;

CREATE TABLE books (
    isbn         VARCHAR(50) PRIMARY KEY,
    book_title   VARCHAR(80),
    category     VARCHAR(30),
    rental_price DECIMAL(10,2),
    status       VARCHAR(10),  -- yes/no
    author       VARCHAR(30),
    publisher    VARCHAR(30)
);
```

- ISBN uniquely identifies a book.
- `rental_price` enables revenue tracking.

This table acts like inventory + pricing — both operations and business.

Create Table: issued_status

```sql
DROP TABLE IF EXISTS issued_status;

CREATE TABLE issued_status (
    issued_id        VARCHAR(10) PRIMARY KEY,
    issued_member_id VARCHAR(30),
    issued_book_name VARCHAR(80),
    issued_date      DATE,
    issued_book_isbn VARCHAR(50),
    issued_emp_id    VARCHAR(10),
    FOREIGN KEY (issued_member_id) REFERENCES members(member_id),
    FOREIGN KEY (issued_emp_id)    REFERENCES employees(emp_id),
    FOREIGN KEY (issued_book_isbn) REFERENCES books(isbn)
);
```

- Connects member + book + employee at issue time.
- This is the core transaction log.

Issued status is where the library "movement" gets recorded.

Create Table: return_status

```sql
DROP TABLE IF EXISTS return_status;

CREATE TABLE return_status (
    return_id        VARCHAR(10) PRIMARY KEY,
    issued_id        VARCHAR(30),
    return_book_name VARCHAR(80),
    return_date      DATE,
    return_book_isbn VARCHAR(50),
    FOREIGN KEY (return_book_isbn) REFERENCES books(isbn)
);
```

- Tracks return events.
- Joins allow finding "not returned yet" books.

Return status closes the loop — without this, overdue and pending tracking isn't possible.

I created the tables (structure) first, and then imported the data into PostgreSQL in one go—no manual row-by-row INSERTs.

Made with GAMMA

CRUD + ANALYSIS

Insert a New Book Record

```sql
INSERT INTO books (isbn, book_title, category,
rental_price, status, author, publisher)
VALUES
('978-1-60129-456-2', 'To Kill a Mockingbird',
'Classic', 6.00, 'yes', 'Harper Lee', 'J.B.
Lippincott & Co.');
```

- Adds a new record into books.
- Values must match column order + data types.

Adding inventory means more options for members and more potential revenue.

Update Member Address

```sql
UPDATE members
SET member_address = '125 Oak St'
WHERE member_id = 'C103';
```

- Updates only one member using WHERE.
- Avoids accidental full-table updates.

Clean member data improves reliability of reports and follow-ups.

Delete a Record from issued_status

```sql
DELETE FROM issued_status
WHERE issued_id = 'IS121';
```

- Deletes one transaction row.
- Used only for corrections.

Removing transaction history changes reporting — so this must be handled carefully.

Made with GAMMA

Retrieve All Books Issued by a Specific Employee

```sql
SELECT *
FROM issued_status
WHERE issued_emp_id = 'E101';
```

issued_id [PK] character varying (10)	issued_member_id character varying (30)	issued_book_name character varying (80)	issued_date date	issued_book_isbn character varying (50)	issued_emp_id character varying (10)
IS130	C106	Moby Dick	2024-04-03	978-0-451-52994-2	E101
IS131	C106	To Kill a Mockingbird	2024-04-04	978-0-06-112008-4	E101
IS154	C105	The Road	2025-12-24	978-0-375-50167-0	E101

- Filters transactions by employee.
- Helps track workload.

This quickly shows who is handling more issue operations.

Made with GAMMA

Members Who Issued More Than One Book

```sql
SELECT
    issued_member_id,
    COUNT(*) AS total_issues
FROM issued_status
GROUP BY issued_member_id
HAVING COUNT(*) > 1;
```

issued_emp_id character varying (10)	count bigint
E105	4
E101	3
E107	4
E102	2

Total rows: 9

- `GROUP BY` counts issues per member.
- `HAVING` filters aggregated results.

This identifies repeat borrowers — the most engaged members.

Made with GAMMA

Create Summary Table (Book-wise Issue Count)

```sql
CREATE TABLE book_issued_cnt AS
SELECT
    b.isbn,
    b.book_title,
    COUNT(ist.issued_id) AS issue_count
FROM issued_status AS ist
JOIN books AS b
ON ist.issued_book_isbn = b.isbn
GROUP BY b.isbn, b.book_title;
```

isbn character varying (50)	book_title character varying (80)	issue_count bigint
978-0-393-05081-8	A Peoples History of the United States	1
978-0-06-440055-8	Charlotte's Web	1
978-0-19-280551-1	The Guns of August	1
978-0-670-81302-4	The Road	1

Total rows: 31

- CTAS creates a ready report table.
- Shows demand per book.

This helps spot which books are popular and need more copies.

Made with GAMMA

Books in a Specific Category

```
SELECT *
FROM books
WHERE category = 'Classic';
```

isbn [PK] character varying (50)	book_title character varying (80)	category character varying (30)	rental_price numeric (10,2)	status character varying (10)	author character varying (30)	publisher character varying (30)
978-0-553-29698-2	The Catcher in the Rye	Classic	7.00	yes	J.D. Salinger	Little, Brown and Compa...
978-0-330-25864-8	Animal Farm	Classic	5.50	yes	George Orwell	Penguin Books
978-0-525-47535-5	The Great Gatsby	Classic	8.00	yes	F. Scott Fitzgerald	Scribner
978-0-141-44171-6	Jane Eyre	Classic	4.00	yes	Charlotte Bronte	Penguin Classics

Total rows: 9

- Filters by category using `WHERE`.
- Useful for category-wise analysis.

Category-level filtering supports better organizing and recommendations.

Made with GAMMA

Total Rental Income by Category

```sql
SELECT
    b.category,
    SUM(b.rental_price) AS total_rental_income,
    COUNT(*) AS total_issues
FROM issued_status AS ist
JOIN books AS b
ON b.isbn = ist.issued_book_isbn
GROUP BY b.category;
```

category character varying (30)	sum numeric	count bigint
History	49.50	7
Fantasy	28.50	4
Dystopian	32.50	5
Horror	7.00	1

Total rows: 10

- Joins issues with books to calculate revenue.
- SUM = money, COUNT = volume.

This reveals which categories bring revenue, not just activity.

Made with GAMMA

Members Registered in the Last 180 Days

```sql
SELECT *
FROM members
WHERE reg_date >= CURRENT_DATE - INTERVAL '180 days';
```

- Finds recent registrations using date logic.
- Supports growth tracking.

This shows new members who can be nurtured into active borrowers.

Made with GAMMA

Employees with Branch + Branch Manager Details

```sql
SELECT
    e1.emp_id,
    e1.emp_name,
    e1.position,
    e1.salary,
    b.branch_id,
    b.manager_id,
    b.branch_address,
    b.contact_no,
    e2.emp_name AS manager_name
FROM employees AS e1
JOIN branch AS b
ON e1.branch_id = b.branch_id
JOIN employees AS e2
ON e2.emp_id = b.manager_id;
```

emp_id character varying (10)	emp_name character varying (30)	position character varying (30)	salary numeric (10,2)	branch_id character varying (10)	manager_id character varying (10)	branch_address character varying (30)	contact_no character varying (15)
E101	John Doe	Clerk	60000.00	B001	E109	123 Main St	+919099988676
E102	Jane Smith	Clerk	45000.00	B002	E109	456 Elm St	+919099988677
E103	Mike Johnson	Librarian	55000.00	B001	E109	123 Main St	+919099988676
E104	Emily Davis	Assistant	40000.00	B001	E109	123 Main St	+919099988676

Total rows: 11

- Combines employee + branch + manager details.
- Uses join back to employees table for manager name.

This creates a clear reporting view of who works where and under which manager.

Made with GAMMA

Books with Rental Price Above Threshold

```sql
CREATE TABLE expensive_books AS
SELECT *
FROM books
WHERE rental_price > 7.00;
```

isbn character varying (50)	book_title character varying (80)	category character varying (30)	rental_price numeric (10,2)	status character varying (10)	author character varying (30)	publisher character varying (30)
978-0-525-47535-5	The Great Gatsby	Classic	8.00	yes	F. Scott Fitzgerald	Scribner
978-0-7432-4722-4	The Da Vinci Code	Mystery	8.00	yes	Dan Brown	Doubleday
978-0-09-957807-9	A Game of Thrones	Fantasy	7.50	yes	George R.R. Martin	Bantam
978-0-393-05081-8	A Peoples History of the United States	History	9.00	yes	Howard Zinn	Harper Perennial

Total rows: 7

- CTAS creates a filtered subset.
- Useful for premium pricing review.

This separates high-value inventory for pricing and demand decisions.

Made with GAMMA

Books Not Yet Returned

```sql
SELECT
    ist.*
FROM issued_status AS ist
LEFT JOIN return_status AS rs
ON rs.issued_id = ist.issued_id
WHERE rs.return_id IS NULL;
```

issued_id character varying (10)	issued_member_id character varying (30)	issued_book_name character varying (80)	issued_date date	issued_book_isbn character varying (50)	issued_emp_id character varying (10)	return_id character varying (10)
IS130	C106	Moby Dick	2024-04-03	978-0-451-52994-2	E101	[null]
IS128	C105	A Tale of Two Cities	2024-04-01	978-0-14-027526-3	E110	[null]
IS134	C107	The Diary of a Young Girl	2024-04-07	978-0-375-41398-8	E106	[null]
IS129	C105	The Stand	2024-04-02	978-0-7434-7679-3	E110	[null]

Total rows: 24

- `LEFT JOIN` keeps all issued records.
- Null on return side means pending return.

This gives a live view of books currently outside the library.

Made with GAMMA

Identify Members with Overdue Books (30-day policy)

```sql
SELECT
    ist.issued_member_id,
    m.member_name,
    bk.book_title,
    ist.issued_date,
    (CURRENT_DATE - ist.issued_date) AS overdue_days
FROM issued_status AS ist
JOIN members AS m
ON m.member_id = ist.issued_member_id
JOIN books AS bk
ON bk.isbn = ist.issued_book_isbn
LEFT JOIN return_status AS rs
ON rs.issued_id = ist.issued_id
WHERE rs.return_date IS NULL
  AND (CURRENT_DATE - ist.issued_date) > 30
ORDER BY overdue_days DESC;
```

issued_member_id character varying (30)	member_name character varying (30)	book_title character varying (80)	issued_date date	over_dues_days integer
C102	Bob Smith	Fahrenheit 451	2024-03-26	673
C103	Carol Davis	Dune	2024-03-27	672
C104	Dave Wilson	Where the Wild Things Are	2024-03-28	671
C105	Eve Brown	Beloved	2024-03-31	668

Total rows: 21

- Finds not-returned books older than 30 days.
- Sorts by highest overdue first.

This produces an actionable overdue list to reduce losses and improve returns.

Made with GAMMA

Branch Performance Report (CTAS)

```sql
CREATE TABLE branch_reports AS
SELECT
    b.branch_id,
    b.manager_id,
    COUNT(ist.issued_id) AS number_book_issued,
    COUNT(rs.return_id) AS number_of_book_return,
    SUM(bk.rental_price) AS total_revenue
FROM issued_status AS ist
JOIN employees AS e
ON e.emp_id = ist.issued_emp_id
JOIN branch AS b
ON e.branch_id = b.branch_id
LEFT JOIN return_status AS rs
ON rs.issued_id = ist.issued_id
JOIN books AS bk
ON ist.issued_book_isbn = bk.isbn
GROUP BY b.branch_id, b.manager_id;
```

branch_id character varying (10)	manager_id character varying (10)	number_book_issued bigint	number_of_book_return bigint	total_revenue numeric
B004	E110	5	3	33.50
B002	E109	2	0	12.00
B003	E109	3	0	21.00
B001	E109	18	8	118.50

Total rows: 5

- Aggregates performance per branch.
- Combines volume + returns + revenue.

This report shows which branches generate activity and revenue.

Made with GAMMA

Active Members (Last 2 Months) using CTAS

```sql
CREATE TABLE active_members AS
SELECT *
FROM members
WHERE member_id IN (
    SELECT DISTINCT issued_member_id
    FROM issued_status
    WHERE issued_date >= CURRENT_DATE - INTERVAL '2
month'
);
```

member_id character varying (10)	member_name character varying (30)	member_address character varying (30)	reg_date date
C105	Eve Brown	890 Maple St	2021-09-...
C106	Frank Thomas	234 Cedar St	2021-10-...
C118	Sam	133 Pine St	2024-06-...
C119	John	143 Main St	2024-05-...

Total rows: 4

- Subquery finds members with recent activity.
- CTAS saves the active segment.

This defines the active user base — perfect for retention and engagement tracking.

Made with GAMMA

Top 3 Employees with Most Book Issues Processed

```sql
SELECT
    e.emp_name,
    b.branch_id,
    b.branch_address,
    COUNT(ist.issued_id) AS books_processed
FROM issued_status AS ist
JOIN employees AS e
ON e.emp_id = ist.issued_emp_id
JOIN branch AS b
ON e.branch_id = b.branch_id
GROUP BY e.emp_name, b.branch_id, b.branch_address
ORDER BY books_processed DESC
LIMIT 3;
```

emp_name character varying (30)	branch_id character varying (10)	manager_id character varying (10)	branch_address character varying (30)	contact_no character varying (15)	no_book_issued bigint
Jessica Taylor	B004	E110	567 Pine St	+919099988679	5
John Doe	B001	E109	123 Main St	+919099988676	3
Mike Johnson	B001	E109	123 Main St	+919099988676	1
Laura Martinez	B005	E110	890 Maple St	+919099988680	6

Total rows: 10

- Counts issues per employee.
- Returns top 3.

This highlights operational performers and helps compare branch workloads.

Made with GAMMA

Conclusion

This project showcases practical SQL skills by building a complete **Library Management System** from scratch. It covers **database creation, table design with relationships, and day-to-day data operations (CRUD)**, along with **CTAS-based reporting** and **advanced queries** for insights like overdue tracking, active members, and branch-level performance. Overall, it provides a strong foundation in **relational database management** and **analysis-driven querying**, similar to what's used in real-world systems.

Made with GAMMA

Thank You ✨

Fiza | Data Analyst Journey

Made with GAMMA